Cameron Lewis

CAP4611

Nov 7th 2025

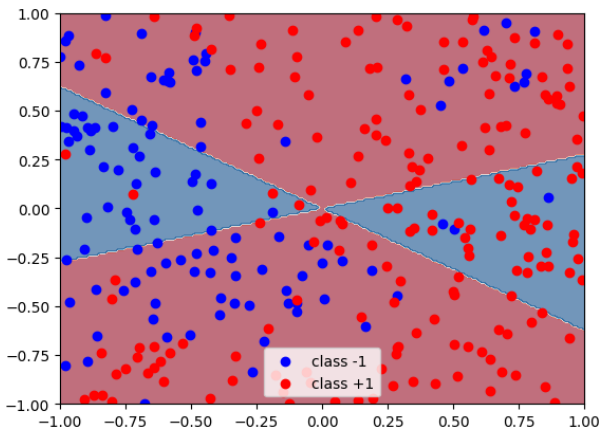
5593788

Assignment 5

1. Kernal Logic Regression
   1.1. Polynomial Kernel (p=2):

   Training error 40.3%

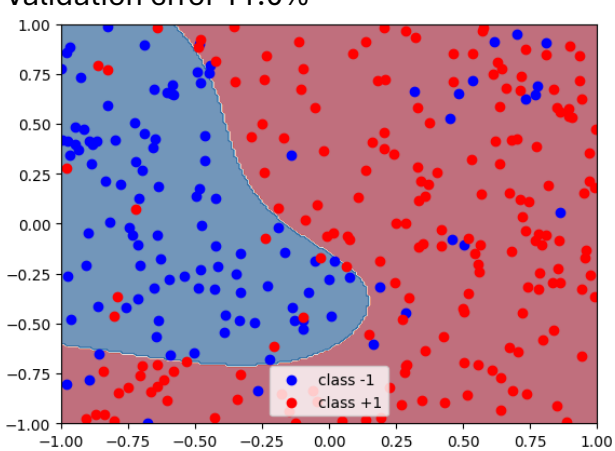
   Validation error 34.0%



   Gaussian RBF Kernel (σ=0.5):

   Training error 12.7%

   Validation error 11.0%



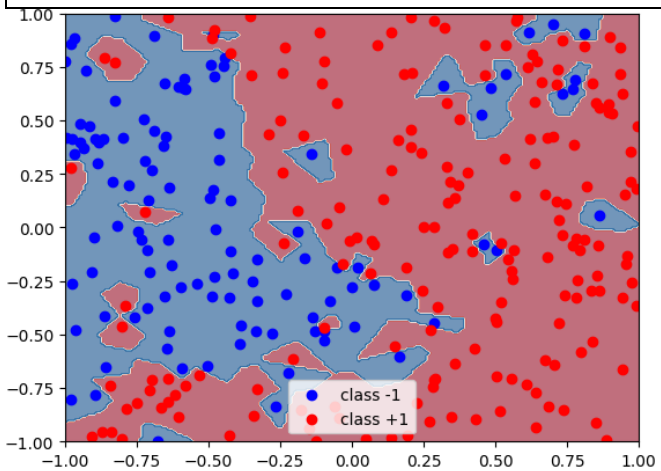
   1.2.

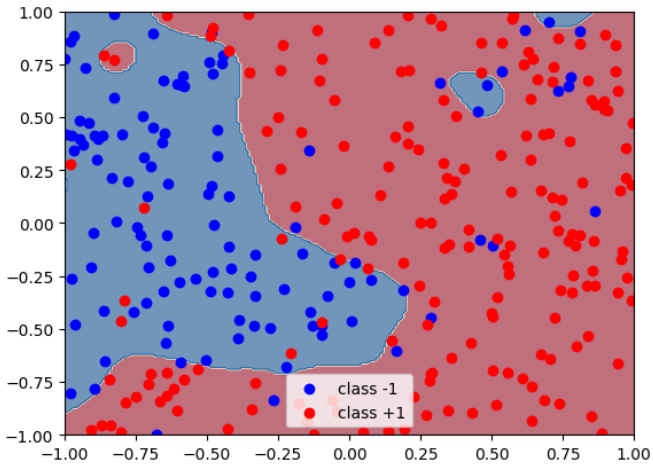
   Best training: σ=0.01, λ=0.0001, error=0.0%

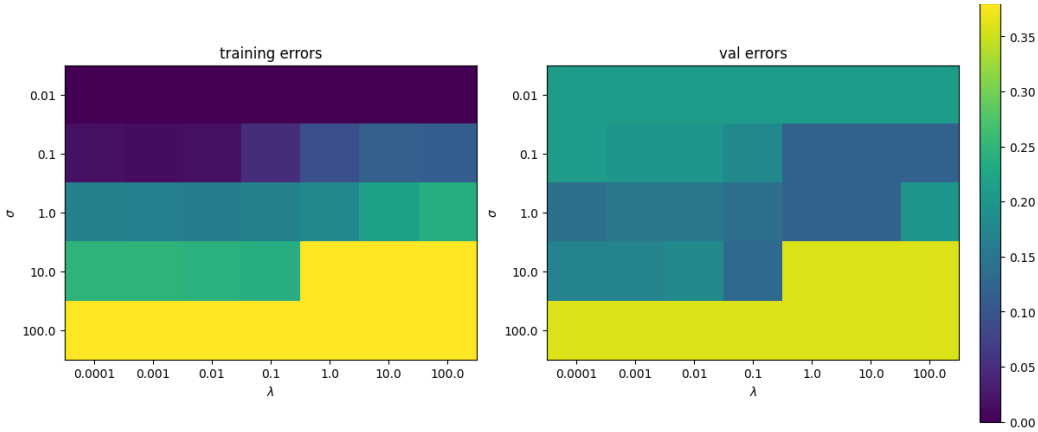
   Best validation: σ=0.1, λ=1.0, error=12.0%

```
for i, sigma in enumerate(sigmas):
    for j, lammy in enumerate(lammys):
        kernel = GaussianRBFKernel(sigma=sigma)
        loss_fn = KernelLogisticRegressionLossL2(lammy=lammy)
        optimizer = GradientDescentLineSearch()
        model = KernelClassifier(loss_fn, optimizer, kernel)
        model.fit(X_train, y_train)

        train_errs[i, j] = np.mean(model.predict(X_train) != y_train)
        val_errs[i, j] = np.mean(model.predict(X_val) != y_val)

    # Find best hyperparameters
best_train_idx = np.unravel_index(np.argmin(train_errs), train_errs.shape)
best_val_idx = np.unravel_index(np.argmin(val_errs), val_errs.shape)

print(
    f"Best training: σ={sigmas[best_train_idx[0]]},
λ={lammys[best_train_idx[1]]}, error={train_errs[best_train_idx]:.1%}")
print(
    f"Best validation: σ={sigmas[best_val_idx[0]]},
λ={lammys[best_val_idx[1]]}, error={val_errs[best_val_idx]:.1%}")

# Train and plot model with best training error hyperparameters
best_train_sigma = sigmas[best_train_idx[0]]
best_train_lammy = lammys[best_train_idx[1]]
kernel = GaussianRBFKernel(sigma=best_train_sigma)
loss_fn = KernelLogisticRegressionLossL2(lammy=best_train_lammy)
optimizer = GradientDescentLineSearch()
model_best_train = KernelClassifier(loss_fn, optimizer, kernel)
model_best_train.fit(X_train, y_train)

fig = plot_classifier(model_best_train, X_train, y_train)
savefig("logRegRBF_best_train.png", fig)
plt.close(fig)

# Train and plot model with best validation error hyperparameters
best_val_sigma = sigmas[best_val_idx[0]]
best_val_lammy = lammys[best_val_idx[1]]
kernel = GaussianRBFKernel(sigma=best_val_sigma)
loss_fn = KernelLogisticRegressionLossL2(lammy=best_val_lammy)
optimizer = GradientDescentLineSearch()
model_best_val = KernelClassifier(loss_fn, optimizer, kernel)
model_best_val.fit(X_train, y_train)
```

```
fig = plot_classifier(model_best_val, X_train, y_train)
savefig("logRegRBF_best_val.png", fig)
plt.close(fig)
```







1.3. The smallest σ and λ values minimize training error, which is exactly what id expect and The parameters demonstrate normal bias-variance tradeoff. The gap between values is a sign of overfitting though

2. Map Estimation

2.1. $f(w) = \frac{1}{2}\sum_i \frac{(w^T x_i - y_i)^2}{\sigma_i^2} + \lambda\|w\|_1$

2.2. $f(w) = \frac{1}{2}\|Xw - y\|_1 + \frac{1}{2\sigma^2}\|w\|^2$

2.3. $f(w) = \frac{\nu+1}{2}\sum_i log(1 + \frac{(w^T x_i - y_i)^2}{\nu}) + \frac{\lambda}{2}\|w - \mu\|^2$

2.4. $f(w) = \sum_i [exp(w^T x_i) - y_i w^T x_i]$

3. Principle Component Analysis

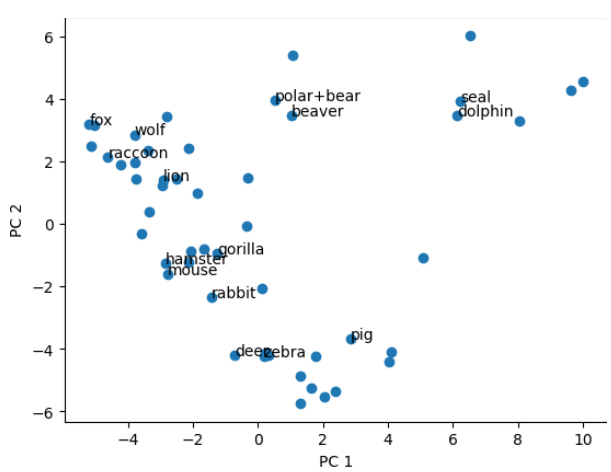
3.1. $[\frac{1}{\sqrt{6}}, -\frac{2}{\sqrt{6}}, \frac{1}{\sqrt{6}}]$

0

$\frac{11}{6}$

3.2.

```
pca = PCAEncoder(k=2)
pca.fit(X_train_standardized)
Z = pca.encode(X_train_standardized)

fig, ax = plt.subplots()
ax.scatter(Z[:, 0], Z[:, 1])
for i in random_is:
    ax.annotate(animal_names[i], xy=Z[i, :])
ax.set_xlabel('PC 1')
ax.set_ylabel('PC 2')
savefig("animals_pca.png", fig)

# Find traits with largest influence
print(f"Largest influence on PC1:
{trait_names[np.argmax(np.abs(pca.W[0, :]))]}")
print(f"Largest influence on PC2:
{trait_names[np.argmax(np.abs(pca.W[1, :]))]}")
```



PC1: furry

PC2: grazer

3.3. Variance explained by k=2: 30.19%

Number of PCs required for 50% variance: 5

4. Stochastic Gradient Descent

4.1.

```
print("Reference:")
loss_fn = LeastSquaresLoss()
optimizer = GradientDescentLineSearch()
model = LinearModel(loss_fn, optimizer, check_correctness=False)
model.fit(X_train, y_train)
print(f"Training MSE: {((model.predict(X_train) - y_train) **
2).mean():.3f}")
print(f"Validation MSE: {((model.predict(X_val) - y_val) **
2).mean():.3f}")

# SGD with different batch sizes
batch_sizes = [1, 10, 100]

for batch_size in batch_sizes:
    print(f"\nSGD with batch_size={batch_size}:")

    loss_fn = LeastSquaresLoss()
    base_optimizer = GradientDescent(max_evals=1)  # only 1 step per
batch
    lr_getter = ConstantLR(multiplier=0.0003)
    optimizer = StochasticGradient(
        base_optimizer=base_optimizer,
        learning_rate_getter=lr_getter,
        batch_size=batch_size,
        max_evals=10  # 10 epochs
    )

    model = LinearModel(loss_fn, optimizer, check_correctness=False)
    model.fit(X_train, y_train)

    train_mse = ((model.predict(X_train) - y_train) ** 2).mean()
    val_mse = ((model.predict(X_val) - y_val) ** 2).mean()

    print(f"Training MSE: {train_mse:.3f}")
    print(f"Validation MSE: {val_mse:.3f}")
```

for batch size 1 and 10 it stayed the same and for 100 it became

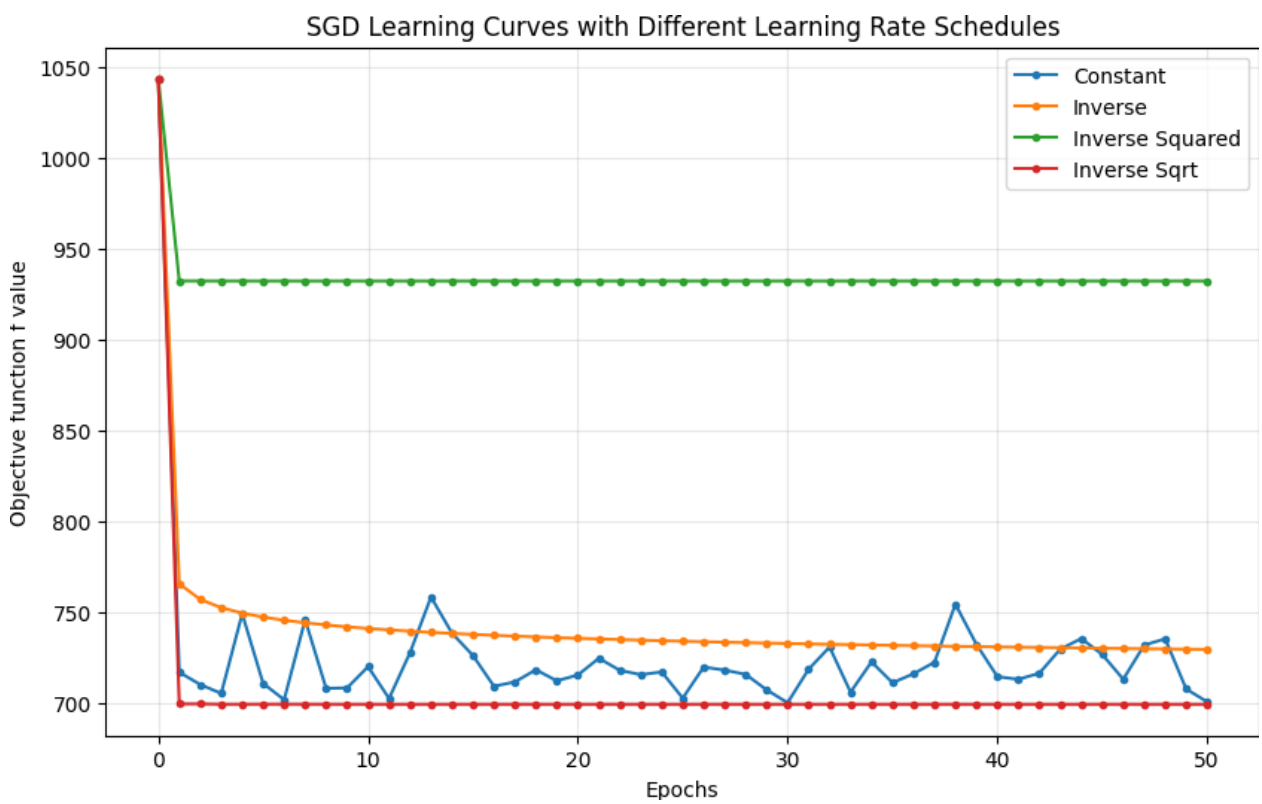
Training MSE: 0.178

Validation MSE: 0.177

4.2.

```python
class ConstantLR(LearningRateGetter):
    def get_learning_rate(self):
        """YOUR CODE HERE FOR Q4.2"""
        self.num_evals += 1
        return self.multiplier
class InverseLR(LearningRateGetter):
    def get_learning_rate(self):
        """YOUR CODE HERE FOR Q4.2"""
        self.num_evals += 1
        return self.multiplier / self.num_evals


class InverseSquaredLR(LearningRateGetter):
    def get_learning_rate(self):
        """YOUR CODE HERE FOR Q4.2"""
        self.num_evals += 1
        return self.multiplier / (self.num_evals ** 2)


class InverseSqrtLR(LearningRateGetter):
    def get_learning_rate(self):
        """YOUR CODE HERE FOR Q4.2"""
        self.num_evals += 1
        return self.multiplier / np.sqrt(self.num_evals)
```

4.3.



Constant and Inverse and Inverse Sqrt

5. Short Answer Questions

5.1. The other normal equations are more efficient when $n >> d$ because matrix inversion becomes computationally cheaper.

5.2. Using kernels allows K-means to find non-linear cluster boundaries.

5.3. In terms of loss functions, MLE uses only the data loss, while MAP adds a regularization term derived from the prior.

5.4. A generative model learns $p(x, y)$ or $p(x \mid y)$ and can generate new data samples, while a discriminative model learns only $p(y \mid x)$ to make predictions.

5.5. Adding a constant to the kernel augments the feature space with a bias term.

5.6. The loss cannot increase since each additional principal component only decreases the reconstruction error.

5.7. If $k > d$, there are more principal components than dimensions. Because there are at most d orthogonal directions of variation, additional components become meaningless.

5.8. A single eigenface is stored as one row of the matrix W.

5.9.

Advantage: SGD scales better with very large datasets because it processes smaller batches rather than the full dataset and uses less memory.

Disadvantage: SGD provides an approximate solution and may converge more slowly than SVD's exact method, and it also requires hyperparameter tuning